# JPMorganChase Model Risk Tech Screen - Python / Data Cleaning

inspect -> clean -> verify -> model -> evaluate

### 1) Load + Inspect Immediately

```
import pandas as pd
import numpy as np

df = pd.read_csv('data.csv')
df = pd.read_parquet('data.parquet')

print(df.head())
print(df.shape)
print(df.columns.tolist())
print(df.dtypes)
df.info()
print(df.describe(include='all'))
```

### 2) Missingness / Duplicates / Types

```
print(df.isna().sum())
print(df.isna().mean().sort_values(ascending=False))
print(df.duplicated().sum())
df = df.drop_duplicates()

df['x'] = pd.to_numeric(df['x'], errors='coerce')
df['date'] = pd.to_datetime(df['date'], errors='coerce')
```

### 3) Missing-Value Handling

```
df['x'] = df['x'].fillna(df['x'].median())
df['x'] = df['x'].fillna(df['x'].mean())
df['x'] = df['x'].ffill()
df['x'] = df['x'].bfill()
df['x'] = df['x'].interpolate()

# ffill/interpolation only when ordering makes sense.
# Never impute blindly; state the assumption.
```

### 4) Sanity Checks / Verification

```
print(df.describe())
print(df['x'].value_counts(dropna=False))
print(df['x'].nunique())
print(df['x'].min(), df['x'].max())
print(df['x'].quantile([.01,.25,.5,.75,.99]))
print(np.isinf(df.select_dtypes('number')).sum())

# Compare before vs after: rows, nulls, min/max,
# distribution, and changed observations.
```

### 5) Ratios + 'Huge Non-Genuine Numbers'

```
df['loan_ratio'] = df['loan_amt'] / df['income']

# Check denominator FIRST:
print((df['income'] <= 0).sum())
df.loc[df['income'] <= 0, 'income'] = np.nan

# Safer:
df['loan_ratio'] = np.where(
    df['income'] > 0,
    df['loan_amt'] / df['income'],
    np.nan
)

print(df['loan_ratio'].describe())
print(df.nlargest(10, 'loan_ratio'))

# Huge values? Check units / zeros / malformed rows.
# Scaling does NOT fix invalid data.
```

### 6) Outliers / Plausibility

```
Q1 = df['x'].quantile(.25)
Q3 = df['x'].quantile(.75)
IQR = Q3 - Q1
lo, hi = Q1 - 1.5*IQR, Q3 + 1.5*IQR
out = df[(df['x'] < lo) | (df['x'] > hi)]

# Do not auto-delete: data error OR genuine tail?
```

### 7) Features / Target / Categorical Data

```
X = df.drop(columns=['default'])
y = df['default']

X = pd.get_dummies(X, drop_first=True)

# Avoid arbitrary integer labels for unordered categories.
```

### 8) Train/Test Split

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Split BEFORE fitting preprocessing to avoid leakage.
```

### 9) Scaling

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test  = scaler.transform(X_test)

# Fit on TRAIN only.
# Helpful for optimization / regularization.
```

### 10) Logistic Regression - Memorize This

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

pred = model.predict(X_test)
prob = model.predict_proba(X_test)[:, 1]
```

### 11) Evaluation

```
from sklearn.metrics import (accuracy_score,
    confusion_matrix, classification_report, roc_auc_score)

print(accuracy_score(y_test, pred))
print(confusion_matrix(y_test, pred))  # [[TN,FP],[FN,TP]]
print(classification_report(y_test, pred))
print(roc_auc_score(y_test, prob))

# Imbalanced classes? Accuracy may mislead.
# Check precision / recall / ROC-AUC.
```

### 12) Model-Risk Checks to Say Out Loud

```
- Features available at prediction time? (leakage)
- Units consistent? dollars vs thousands / % vs decimal
- Ratios economically plausible?
- Did cleaning alter unaffected rows?
- Is missingness itself informative?
- Outliers: errors or genuine risky cases?
- Does class imbalance distort accuracy?

Best line:
'Before scaling or modeling, I want to verify that the
extreme values are genuine rather than data-quality
issues.'
```